First Automation FIN1209: automate price, bars, volume and three oscillators in R

Write a short R script that reads price files and takes five votes a day (the price trend and price bar, then the RSI, the MACD and the stochastic), counts how many agree, checks the volume, and draws the signals. Then draw a flowchart that explains it.

Student worksheet 100 points | a few hours over a week

Benjamin C. Sotelo | Institute of Accounts, Business and Finance, FEU Manila

Name _____ Section _____ Date _____

What you are making

Read this page first. It is short.

You will teach R to read **price, bars and volume** (Chapters 1 to 7) and the three **window oscillators** of Chapter 8. You put a Google Sheets price file in a folder, run a short **R script**, and it works out, day by day, five votes for buy or sell: the **price trend**, the **price bar**, the **RSI**, the **MACD** and the **stochastic**. It counts how many agree, checks the **volume**, and gives a trigger price. It writes a table and a chart. Add new prices and run it again, and both update. That is **automation**.

THIS PROGRAM IS NOT FOOLPROOF

It follows rules from our slides. Its signals can be wrong, and they can lose money. It is a class exercise, not advice.

THE GUIDE VIDEO HAS TWO VERSIONS

Long version from 2:06, short version from 26:59, outputs from 29:04. The long version types every line and explains it. The short version explains each block of code and what it produces. Watch one of the two, then carry on from the outputs.

YOU DO NOT NEED TO KNOW R

Nobody in this class was taught to code. The script is about 60 lines, Part 3 shows what it does, and Part 5 builds it in seven small steps. The numbers it prints are not advice: Part 7 shows what they mean, and why signals that agree may agree for a bad reason.

What you get, and what you hand in

	WHAT
You get	This manual, a video that shows every step on screen, the finished example script and the example flowchart (`.pdf` and `.bpmn`). No price files: you download them yourself.
You hand in	Your own R script and a flowchart of how it works. Your own stock, and your own pair of Chapter 8 oscillators (Part 9).
Due date	_____ (to be filled in: after the Saturday 7 November Chapter 8 class)

WHAT THESE SIGNALS ARE

The output is **the signals of three rules, applied to past prices**. It is not investment advice and it does not say what price will do next. Chapter 1's point still holds: a forecast cannot be checked when it is made.

1. What to install

Free, and about 30 minutes. If you cannot install, use the browser option.

You need two programs. **R** is the language, and it does the work. **RStudio Desktop** is a friendly window for writing and running R. Install R first, then RStudio.

> NO R PACKAGES
> The example needs **no packages at all**. It uses only what comes with R. You never type `install.packages`. Your own script should not need one either, because the marker's computer will not have it.

W Windows

1. Go to **https://cran.r-project.org/bin/windows/base/** and click the **Download R** link at the top. Open the file you downloaded and click Next through the installer, keeping every default.
2. Go to **https://docs.posit.co/ide/user/download.html** and click the big **Download for Windows** button. Open the file and click Next through the installer.
3. Open **RStudio** from the Start menu. A window opens with a pane called Console on the left. That is where R talks.

M Mac

1. Go to **https://cran.r-project.org/bin/macosx/**. Click the first `.pkg` under the latest release, the one for your chip (the Apple icon, then About This Mac, tells you whether it is Apple silicon or Intel). Open the file and click Continue through the installer.
2. Go to **https://docs.posit.co/ide/user/download.html**, click **Download for macOS**, open the file, and drag RStudio into the Applications folder.
3. Open **RStudio** from Applications. A window opens with a pane called Console on the left.

C If you cannot install anything

Use **Posit Cloud**, which runs RStudio in your browser. Go to **https://posit.cloud**, make a free account, then **New Project**, then **New RStudio Project**. Everything in this handout works the same way. To put the example folder in, use the **Upload** button in the Files pane.

F For the flowchart

Draw it in **bpmn.io** at **https://demo.bpmn.io/**. It runs in the browser, is free, and needs no account. Or install **Camunda Modeler** from **https://camunda.com/download/modeler/**. Both draw BPMN and both save a `.bpmn` file and export a picture. If you would rather use Markdown, **https://mermaid.live** draws a Mermaid flowchart in the browser.

2. Where to get price data, and what kind

Checked on the public sites on 4 November 2026. Where a claim needs an account, it says so instead of guessing.

Chapter 1 names the data technical analysis reads: **price, volume and open interest**. For this activity you need one bar per trading day, with six columns.

The script needs six columns: Date, Open, High, Low, Close and Volume. The RSI and the MACD use the close; the stochastic uses the high, low and close; a buy's trigger uses the high and a sell's the low. Volume is not used here (Chapter 6 covers it). **Open interest** exists only for futures, so a stock has none, and no source below gives one for a share.

The free sources, and what we found

SOURCE	OPEN, HIGH, LOW, CLOSE, VOLUME	ACCOUNT	CSV FILE
Google Sheets, GOOGLEFINANCE	Yes, daily or weekly, all five, with the formula `"all"`. The Help page lists them.	A Google account	Yes: File, Download, CSV. **This handout uses it, for US-listed stocks only.** `PSE:JFC` returned `#N/A`, so there is no daily history for Philippine stocks.
PSE EDGE (edge.pse.com.ph), the exchange's own site	Today's open, high, low, last price and volume, and the 52-week range, for each Philippine stock. We found no table of past days.	None to read	No download link found. The site says its content is copyrighted.
Yahoo Finance	A history table of date, open, high, low, close, adjusted close and volume for US stocks, readable without signing in. We found no Philippine stocks.	None to read the table	No Download button when signed out. Whether one appears after signing in is not checked.
Investing.com	Not checked: the site refused our automated visit. It is a common place to look for Philippine stocks, so check what it offers for the one you want.	Not checked	Not checked
TradingView	Charts Philippine stocks, for example PSE:BDO, with a free public page. Export of the chart data was not checked.	None to view; export needs an account	Not checked, needs an account
Your broker's platform, for example COL Financial	Not checked: it sits behind a login.	Yes, a brokerage account	Not checked

Prices in the **adjusted close** column are corrected for splits and dividends. The example does not use it.

The script works with any source whose CSV has the columns of Part 5. Google Sheets is the one route this handout walks through step by step, because every student already has an account and it exports a CSV in the right layout. If you find Philippine data elsewhere and can save it as a CSV with those column names, it will run.

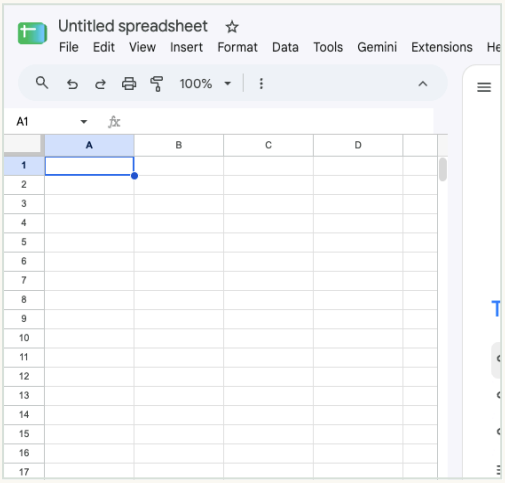
2b. Get the example prices yourself

The same route you will use for your own task. We give you no price files.

Google Sheets can fetch daily prices with one formula, and it gives exactly the columns the contract needs: Date, Open, High, Low, Close, Volume. It works for **US-listed stocks**. You do this once per stock, and you use the same dates as below so your numbers match this handout.

1 Open a blank spreadsheet

Go to **https://sheets.new** while signed in to your Google account. A blank spreadsheet opens. Click cell **A1**.



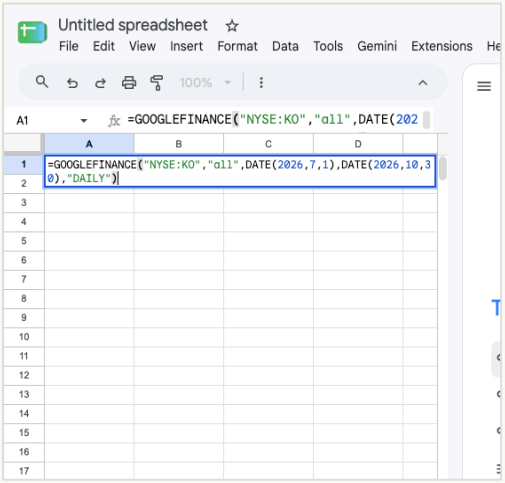
2 Type the formula

In A1 type this formula for Coca-Cola, then press Enter:

TYPE EXACTLY THIS

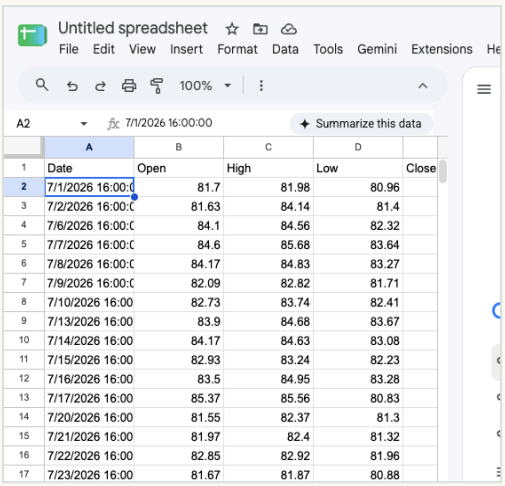
```
=GOOGLEFINANCE("NYSE:KO","all",DATE(2026,7,1),DATE(2026,9,30),"DAILY")
```

The picture shows the formula being typed. Its end date in that capture reads 2026,10,30; **use 2026,9,30** for this first run.



3 Check the result

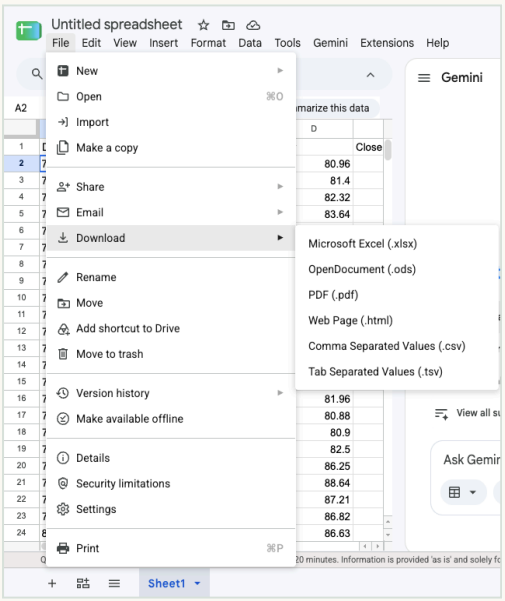
Wait a moment. About 64 rows fill down from A1, with the six column names in row 1. The dates read like `7/1/2026 16:00:00`: that is the layout Google Sheets gives, and the script reads it as it is, so you never edit a date. If you see `#N/A`, check the quote marks are straight and the ticker is spelled `NYSE:KO`.



	Date	Open	High	Low	Close
2	7/1/2026 16:00:0	81.7	81.98	80.96	
3	7/2/2026 16:00:0	81.63	84.14	81.4	
4	7/6/2026 16:00:0	84.1	84.56	82.32	
5	7/7/2026 16:00:0	84.6	85.68	83.64	
6	7/8/2026 16:00:0	84.17	84.83	83.27	
7	7/9/2026 16:00:0	82.09	82.82	81.71	
8	7/10/2026 16:00	82.73	83.74	82.41	
9	7/13/2026 16:00	83.9	84.68	83.67	
10	7/14/2026 16:00	84.17	84.63	83.08	
11	7/15/2026 16:00	82.93	83.24	82.23	
12	7/16/2026 16:00	83.5	84.95	83.28	
13	7/17/2026 16:00	85.37	85.56	80.83	
14	7/20/2026 16:00	81.55	82.37	81.3	
15	7/21/2026 16:00	81.97	82.4	81.32	
16	7/22/2026 16:00	82.85	82.92	81.96	
17	7/23/2026 16:00	81.67	81.87	80.88	

4 Download it as a CSV

Choose **File, Download, Comma Separated Values (.csv)**. The file holds the values, not the formula, so there is nothing more to do to it. Your browser saves a file named after the spreadsheet.



Rename the file **KO.csv** and move it into your `data` folder. The name matters: the script takes the ticker from it. Then do the same for a second stock in a new spreadsheet, with this formula, and save it as **F.csv** in `data`. Part 6 runs the script on these two files.

CELL A1, FOR F (FORD)

```
=GOOGLEFINANCE("NYSE:F","all",DATE(2026,7,1),DATE(2026,9,30),"DAILY")
```

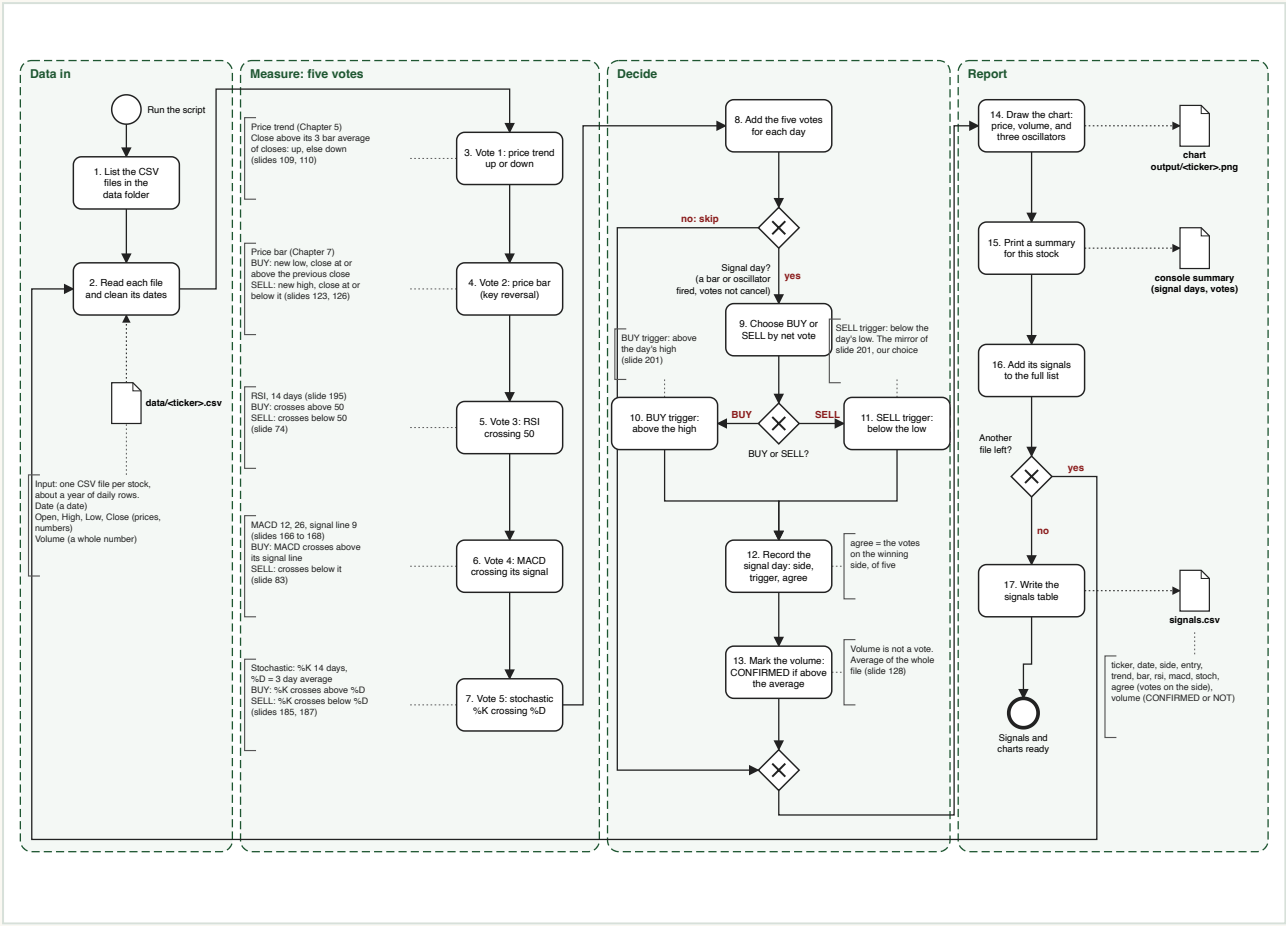
Later, for the update in Part 6: in each spreadsheet change `DATE(2026,9,30)` in the formula to `DATE(2026,10,30)`, download again, and replace the file in `data`, still named KO.csv and F.csv. There is no capture of that second download because it is the same four steps.

SWAP THE TICKER, NOTHING ELSE

To use another US stock, change only the text inside the first quotes of the formula, for example `"NASDAQ:AMD"`. **Philippine stocks do not work here.** We tried `PSE:JFC` and Sheets answered `#N/A`: GOOGLEFINANCE gives no daily history for Philippine stocks. For a PSE stock you need another source from Part 2, for example Investing.com, then save its data as a CSV with the columns of Part 5.

3. What the script does

Input, process, output. A flowchart says it in shapes; yours must match your script line for line.



The example script as a BPMN flowchart. `First Automation FIN1209 - Example Flowchart.bpmn` opens in bpmn.io.

Drawn for this course by build/bpmnkit.py.

Input, process, output

PART	WHAT HAPPENS
Input	A price file per stock in the `data` folder, from Google Sheets: a date, four prices and a volume for each trading day.
Process	For every stock and every day, five votes for buy or sell: the price trend and the price bar (Chapters 5 and 7), then the RSI, the MACD and the stochastic crossing (Chapter 8). A signal day is a day the bar or an oscillator fires and the votes do not cancel. `agree` counts the votes on the winning side, and above-average volume marks the day CONFIRMED.
Output	A table of the signal days with a trigger level, a chart of price, volume and the three oscillators, and a summary line per stock.

Which lines each shape stands for

SHAPE	SAYS	LINES
Start event	Run the script	the moment you click Source
1	List the CSV files in the data folder	line 19
2	Read each file and clean its dates	lines 20 to 22
3	Vote 1: price trend	lines 24 to 26
4	Vote 2: price bar (key reversal)	lines 27 to 29
5	Vote 3: RSI crossing 50	line 30 and line 33
6	Vote 4: MACD crossing its signal line	line 31 and line 34
7	Vote 5: stochastic %K crossing %D	line 32 and line 35
8	Add the five votes for each day	line 36
Gateway	Signal day?	line 37 and line 38
9	Choose BUY or SELL by the net vote	line 39
10, 11	BUY trigger = high, SELL trigger = low	line 41
12	Record the signal day	lines 40 to 44
13	Mark the volume CONFIRMED or NOT	line 45
14	Draw the chart	lines 46 to 59
15	Print a summary for this stock	line 61
16	Add its signals to the full list	line 60
17	Write the signals table	line 63

BPMN needs five shapes: a thin circle starts, a thick circle ends, a rounded rectangle is a task named with a verb, a diamond with an X is a yes or no question with every arrow labelled, and an arrow is the order. A page-shaped data object is a file, and a text annotation says a rule. Every shape on your flowchart should point at lines of your script.

4. The five votes and the volume check

Each rule as the slides give it, with the slide number and title. Chapter 5, 7 and 8 student editions.

What the slides say

SLIDE	RULE
Ch 5: 109, 110 "Drummond geometry", "Reading a PLdot line"	**Trend vote.** A three bar moving average of price: price above it is bullish, below it is bearish.
Ch 7: 123, 126 "The key reversal is when a strong move suddenly turns"	**Bar vote.** A bearish key reversal bar forms a higher high than the previous bar and closes near or below the previous close. The bullish one is the opposite: a new low, then a close near or above.
Ch 7: 128 "Figure 7.15 shows the key reversal on the book's chart"	**Volume check.** The key reversal day is marked with above-average volume. Volume is not a vote: it marks a signal CONFIRMED or not.
Ch 8: 74	**RSI vote.** Crossing above the 50 equilibrium line is a buy and below it a sell.
Ch 8: 83, 166 to 168	**MACD vote.** 12 period EMA minus 26 period EMA, 9 period signal line. Crossing above the signal line is a buy, below it a sell.
Ch 8: 185, 187, 189	**Stochastic vote.** %K over 14 bars, %D its 3 period average. %K crossing above %D is a buy, below it a sell.
Ch 8: 193 to 195	The RSI uses 14 periods and plain averages of gains and losses.
Ch 8: 201	The trigger for entry is normally the high of the bar tied to the buy signal.
Ch 8: 78, 81	Oscillators built from the same prices are highly correlated: multicollinearity, a false consensus.

OUR CHOICES, NOT LIM'S RULE

Trend. Slide 110's own chart averages closes, and the book's PLdot averages typical price, which Chapter 5 does not define. We average the closes. Chapter 2's peaks and troughs trend is not computed.

Key reversal. The slides say "near or below" the previous close. We read it as at or below, and at or above for the bullish one.

Volume. The slides give no period for "average", so it is the average volume of the whole file.

Entry. A buy triggers above the signal day's high, as slide 201 says. The slides give no sell trigger, so a sell triggers below the signal day's low: the mirror of Lim's buy trigger on slide 201, our choice, not Lim's rule. (Lim's own drawing of the ID/NR6 pattern, Chapter 7 slide 207, puts its long trigger at a bar's high and its short trigger at that bar's low, but that is one pattern.) For both sides the price is a trigger level: the trade happens only if price later goes beyond it.

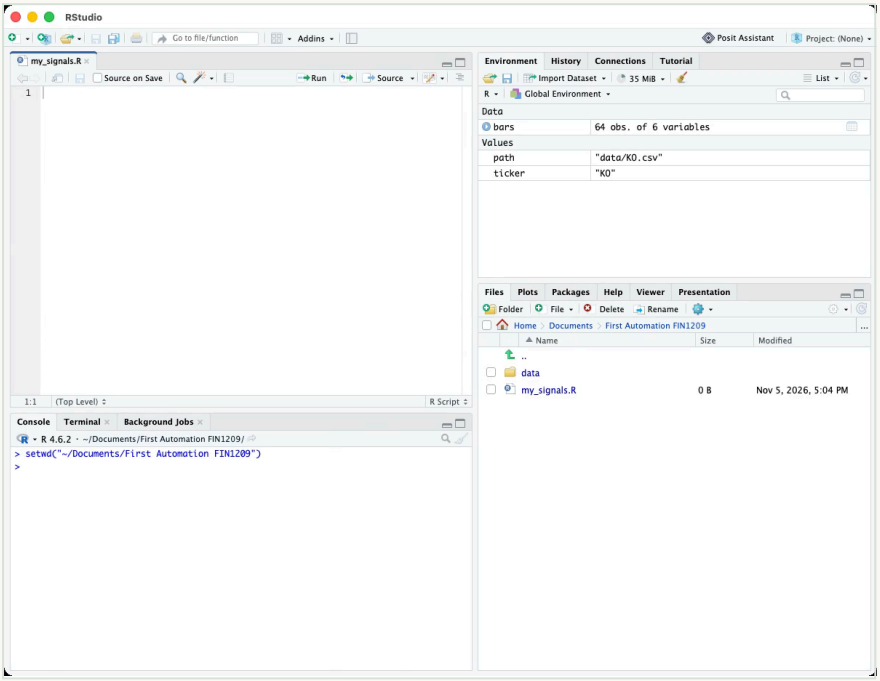
Signal day. Any day the bar or an oscillator fires and the five votes do not cancel. `agree` counts the votes on the winning side.

5. Write the script in seven steps

Each step adds one idea. The new lines are highlighted, and what R printed is under them.

1 Set up

Make a folder `First Automation FIN1209` in your Documents folder with a folder `data` inside it, and put the two Part 2 files `KO.csv` and `F.csv` in `data`. In RStudio choose **File, New File, R Script**, save it in the folder, then **Session, Set Working Directory, To Source File Location**. Type `install.packages("TTR")` in the Console once. Run your script with **Code, Source**.



1 Read every price file

Step one: read every price file in the data folder. Type each group of lines, read what it does, then move on.

What the new lines do (line numbers of the finished script)

LINES	WHAT THEY DO
1 to 6	These six lines are comments. Anything after a hash is a note for people, and R skips it. Ours say what the script does, and which Chapter and slide each vote comes from. Always write down where a rule comes from.
19	This is a loop. `list.files()` looks in the folder called data and gives back the name of every file that ends in `.csv`. For path in means: take the files one at a time, and run everything inside the curly brackets once for each file. The script will work for one stock or twenty.
20 to 21	Ticker is a variable, a name that holds a value. Basename strips the folder from the path, and sub removes the `.csv`, so `KO.csv` becomes KO. Read `.csv` then opens the file as a table, one row per day, and calls it bars.
22	Google Sheets writes dates like 7/1/2026, followed by a time. Sub cuts off everything from the first space. `as.Date()` turns the text into a real date, and `tryFormats` lists the two layouts it may find. Now R can sort and plot by date.

TYPED IN STEP 1, AND THE LINE THAT PRINTS THE RESULT

```
for (path in list.files("data", pattern = "\\.csv$", full.names = TRUE)) {
  ticker <- sub("\\.csv$", "", basename(path))
  bars <- read.csv(path)
  bars$Date <- as.Date(sub(" .*", "", bars$Date), tryFormats = c("%Y-%m-%d", "%m/%d/%Y"))
  print(head(bars, 3))
}
```

R PRINTED

```
        Date  Open  High   Low Close   Volume
1 2026-07-01 13.85 13.88 13.55 13.64 44543988
2 2026-07-02 13.82 14.00 13.22 13.36 49011930
3 2026-07-06 13.51 13.90 13.42 13.83 54301369
        Date  Open  High   Low Close   Volume
1 2026-07-01 81.70 81.98 80.96 81.29 21734809
2 2026-07-02 81.63 84.14 81.40 84.14 18320542
... (1 more lines)
```

Run it with Code, Source. Now run it with Code, then Source. In the Console you should see the first three rows of each file, one after the other: a date, the open, high, low and close prices, and the volume. The dates are now real dates, written year, month, day.

2 Vote 1: the price trend

Step two, the first vote: is the price trending up or down? Type each group of lines, read what it does, then move on.

What the new lines do (line numbers of the finished script)

LINES	WHAT THEY DO
7	R does a lot, but the indicators come in a package called TTR. Library loads it. You install it only once, with `install.packages()`, typed in the Console.
24	SMA is a simple moving average. Three means the average of the last three closes. Chapter 5, slides 109 and 110, call this line the PLdot. Price above it is bullish, and below it bearish.
25	If the close is above the average, the vote is plus one, up. Otherwise it is minus one, down. If else makes that choice for every day at once. A vote is just a number, plus one for buy, minus one for sell.
26	The first two days have no three day average yet, so the vote is missing. `is.na()` finds the missing ones, and we set them to zero, no vote.

TYPED IN STEP 2, AND THE LINE THAT PRINTS THE RESULT

```
library(TTR)
  pldot <- SMA(bars$Close, 3)
  trend_vote <- ifelse(bars$Close > pldot, 1, -1)
  trend_vote[is.na(trend_vote)] <- 0
  print(table(trend_vote))
```

R PRINTED

```
trend_vote
-1  0  1
35  2 27
trend_vote
-1  0  1
32  2 30
```

Run it with Code, Source. Run it. The Console shows a small table with a count for minus one, plus one and zero. Most days are one or the other, and zero appears only on the first two days, which have no three day average yet.

3 Vote 2: the price bar

Step three, the second vote: what is today's bar telling us? Type each group of lines, read what it does, then move on.

What the new lines do (line numbers of the finished script)

LINES	WHAT THEY DO
23	Before gives yesterday's value in today's row. It shifts a column down by one, and the first day gets NA, because it has no yesterday. We need it to compare today's bar with yesterday's.
27 to 28	The key reversal, Chapter 7, slides 123 and 126. A bullish one: today's low is lower than yesterday's, but the close is at or above yesterday's close. That is plus one. Bearish: a higher high, and a close at or below yesterday's close. That is minus one. Otherwise zero. The book says near; we say at or.
29	The first day again has no yesterday, so the bar vote there is missing. Set it to zero.

TYPED IN STEP 3, AND THE LINE THAT PRINTS THE RESULT

```
1   before <- function(x) c(NA, head(x, -1))
2   bar_vote <- ifelse(bars$Low < before(bars$Low) & bars$Close >= before(bars$Close), 1,
3                      ifelse(bars$High > before(bars$High) & bars$Close <= before(bars$Close),
  -1, 0))
4   bar_vote[is.na(bar_vote)] <- 0
5   print(bars$Date[which(bar_vote != 0)])
```

R PRINTED

```
[1] "2026-07-02" "2026-07-13" "2026-07-14" "2026-07-17" "2026-08-03"
[6] "2026-08-11" "2026-08-28" "2026-09-10" "2026-09-25"
 [1] "2026-07-06" "2026-07-17" "2026-07-21" "2026-08-03" "2026-08-07"
 [6] "2026-08-10" "2026-08-12" "2026-08-17" "2026-08-26" "2026-09-01"
[11] "2026-09-02" "2026-09-08" "2026-09-17" "2026-09-21" "2026-09-23"
[16] "2026-09-30"
```

Run it with Code, Source. Run it. The Console lists the dates on which a key reversal bar happened, bullish or bearish. Not every day, only a handful.

4 Votes 3 to 5: the oscillators

Step four, three more votes: the oscillators. Type each group of lines, read what it does, then move on.

What the new lines do (line numbers of the finished script)

LINES	WHAT THEY DO
8 to 15	A function packs lines under a name so we can reuse them. Cross takes two lines of numbers, a and b. It rounds them, so a rounding error is not mistaken for a cross. Then it gives plus one on the day a goes from at or below b to above it, minus one on the day it goes from at or above to below, and zero otherwise. A crossing is the signal.
16	Word turns a vote into text: plus one becomes BUY, minus one SELL, zero nothing. We use it for the table.
30	The RSI, the relative strength index. The input is the close. `n = 14` means it looks at the last fourteen days. M A type SMA means plain averages, as slide 195 says. It swings between zero and one hundred.
31	The MACD is a twelve day average minus a twenty six day average, and its signal line is a nine day average of that, slides 166 to 168. N fast, n slow and n sig are those three numbers. Percent false keeps it as a plain difference.
32	The stochastic uses the high, low and close. Fast K, fourteen days, says where the close sits within the last fourteen days' range. Fast D, three, is a three day average of it, slide 187.
33 to 35	Now find the crossings. The RSI crossing its fifty line, slide 74. The MACD crossing its signal line, slide 83. The stochastic K crossing D, slide 187. Each result is plus one, minus one, or zero.

TYPED IN STEP 4, AND THE LINE THAT PRINTS THE RESULT

```
1  cross <- function(a, b) {        # +1 on the day a crosses above b, -1 on the day it crosses
   below
2    a <- round(a, 6)
3    b <- round(rep(b, length.out = length(a)), 6)     # rounding stops a rounding-error tie from
   counting as a cross
4    s <- ifelse(a > b & before(a) <= before(b), 1, ifelse(a < b & before(a) >= before(b), -1, 0))
5    s[is.na(s)] <- 0
6    s
7  word <- function(s) ifelse(s == 1, "BUY", ifelse(s == -1, "SELL", ""))
8    before <- function(x) c(NA, head(x, -1))
9    rsi <- RSI(bars$Close, n = 14, maType = "SMA")
10   macd <- MACD(bars$Close, nFast = 12, nSlow = 26, nSig = 9, percent = FALSE)
11   sto <- stoch(bars[, c("High", "Low", "Close")], nFastK = 14, nFastD = 3)
12   rsi_sig <- cross(rsi, 50)
13   macd_sig <- cross(macd[, "macd"], macd[, "signal"])
14   stoch_sig <- cross(sto[, "fastK"], sto[, "fastD"])
15   print(colSums(abs(cbind(rsi_sig, macd_sig, stoch_sig))))
16 }
```

R PRINTED

```
  rsi_sig  macd_sig stoch_sig
        5         2        23
  rsi_sig  macd_sig stoch_sig
        7         2        20
```

Run it with Code, Source. Run it. The Console shows, for each stock, how many times each oscillator crossed. The stochastic crosses most often, the MACD least. These are just counts of crossings.

5 Count the votes and keep the signal days

Step five: add up the votes, and keep the signal days in a table. Type each group of lines, read what it does, then move on.

What the new lines do (line numbers of the finished script)

LINES	WHAT THEY DO
17 to 18	All signals will collect every stock's signals. `dir.create()` makes the output folder if it is not there. Both go before the loop, because we only want them once.
36	Cbind puts the five votes side by side, one column each, one row per day. Rows sums will add them up for each day.
37 to 38	A signal day is a day the bar or an oscillator fired, written with the vertical bar for or, and the votes do not cancel out. Which gives the row numbers of those days.
39	If the votes add up above zero the side is BUY, otherwise SELL.
40 to 41	Now the table. `data.frame()` builds it, one row for each signal day. Ticker, date and side. The entry is a trigger level. A buy triggers above the signal day's high, as Lim says on slide 201. A sell triggers below the signal day's low. That is the mirror of Lim's buy trigger, our choice, because the slides give no sell trigger. The trade happens only if price later goes beyond the level.
42 to 45	Then what each vote said, written as words, and agree: how many of the five votes are on the winning side. And the volume: if the day's volume is above the average volume of the file, it is CONFIRMED, as slide 128 marks the key reversal day. Volume is not a vote. It says how much weight the day carried.
60	R bind adds this stock's table under all the others.
63	After the loop, write `.csv` saves the whole table as `signals.csv` in the output folder.

TYPED IN STEP 5, AND THE LINE THAT PRINTS THE RESULT

```
 1 all_signals <- NULL
 2 dir.create("output", showWarnings = FALSE)
 3   votes <- cbind(trend_vote, bar_vote, rsi_sig, macd_sig, stoch_sig)
 4   fired <- bar_vote != 0 | rsi_sig != 0 | macd_sig != 0 | stoch_sig != 0
 5   day <- which(fired & rowSums(votes) != 0)
 6   side <- ifelse(rowSums(votes)[day] > 0, "BUY", "SELL")
 7   signals <- data.frame(ticker = ticker, date = bars$Date[day], side = side,
 8                         entry = ifelse(side == "BUY", bars$High[day], bars$Low[day]),   #
   trigger: BUY above the high, SELL below the low
 9                         trend = word(trend_vote[day]), bar = word(bar_vote[day]), rsi =
   word(rsi_sig[day]),
10                         macd = word(macd_sig[day]), stoch = word(stoch_sig[day]),
11                         agree = rowSums(votes[day, , drop = FALSE] == ifelse(side == "BUY", 1,
   -1)),
12                         volume = ifelse(bars$Volume[day] > mean(bars$Volume), "CONFIRMED",
   "NOT"))
13   all_signals <- rbind(all_signals, signals)
14   print(tail(signals, 3))
15 write.csv(all_signals, "output/signals.csv", row.names = FALSE, quote = FALSE)
```

R PRINTED

```
   ticker       date side entry trend bar rsi macd stoch agree    volume
26      F 2026-09-18 SELL 12.94  SELL              SELL     2 CONFIRMED
27      F 2026-09-25  BUY 12.81  SELL BUY           BUY      2       NOT
28      F 2026-09-28 SELL 12.29  SELL              SELL     2       NOT
   ticker       date side entry trend  bar  rsi macd stoch agree volume
28     KO 2026-09-23 SELL 87.61   BUY SELL SELL               2    NOT
29     KO 2026-09-24 SELL 88.10  SELL                SELL     2    NOT
... (1 more lines)
```

Run it with Code, Source. Run it. The Console shows the last three rows of the signals table. Each row is a signal day, with the side, the trigger level, each vote, how many agree, and whether the volume confirmed it. It is also saved as `signals.csv`.

6 Draw the chart

Step six: draw the chart. Type each group of lines, read what it does, then move on.

What the new lines do (line numbers of the finished script)

LINES	WHAT THEY DO
46 to 48	Png opens a picture file in the output folder, named after the stock. Layout splits it into five panels, the first three times as tall as the others. Par sets the margins.
49 to 50	Plot draws the closing price as a line, against the date. Lines adds the orange three day average on top of it.
51 to 53	Now mark the signals. Big means three or more votes agree. Points puts a triangle at each signal's trigger level, green up for a buy, red down for a sell, and bigger when three or more agree. Text numbers the big ones with their row in the table.
54 to 55	The second panel is the volume as bars, gold on the days confirmed by volume, with a dashed line at the average.
56 to 58	Then one panel for each oscillator: the RSI with its fifty line, the MACD with its signal line, and the stochastic.
59	`dev.off()` closes the picture file. Until then nothing is saved.

TYPED IN STEP 6, AND THE LINE THAT PRINTS THE RESULT

```
  png(file.path("output", paste0(ticker, ".png")), width = 1200, height = 1200)
  layout(matrix(1:5), heights = c(3, 1, 1, 1, 1))
  par(mar = c(2, 4, 3, 1))
  plot(bars$Date, bars$Close, type = "l", xlab = "", ylab = "Close", main = paste(ticker, "-
price, bars, volume and oscillators"))
  lines(bars$Date, pldot, col = "orange")
  big <- signals$agree >= 3
  points(signals$date, signals$entry, pch = ifelse(side == "BUY", 24, 25), bg = ifelse(side ==
"BUY", "darkgreen", "red"), cex = ifelse(big, 1.8, 0.7))
  if (any(big)) text(signals$date[big], signals$entry[big], labels = which(big), pos = 3)
  plot(bars$Date, bars$Volume, type = "h", xlab = "", ylab = "Volume", col = ifelse(bars$Date
%in% signals$date[signals$volume == "CONFIRMED"], "gold", "gray"))
  abline(h = mean(bars$Volume), lty = 2)
  plot(bars$Date, rsi, type = "l", xlab = "", ylab = "RSI"); abline(h = 50, lty = 2)
  matplot(bars$Date, macd, type = "l", lty = 1, xlab = "", ylab = "MACD"); abline(h = 0, lty =
2)
  matplot(bars$Date, sto[, c("fastK", "fastD")], type = "l", lty = 1, xlab = "", ylab =
"Stochastic")
  dev.off()
```

Run it with Code, Source. Run it. Nothing prints, because the chart goes into a picture file. In the output folder there is now one picture for each stock. Here is the KO chart.

7 A summary line for each stock

Step seven: print one summary line for each stock. Type each group of lines, read what it does, then move on.

What the new lines do (line numbers of the finished script)

LINES	WHAT THEY DO
61	Cat prints a line in the Console. It says the stock, how many signal days, how many had three or more votes agreeing, and how many were on above-average volume. The numbers come straight from the table.

TYPED IN STEP 7, AND WHAT R PRINTED

```
  cat(ticker, ":", nrow(signals), "signal days,", sum(signals$agree >= 3), "with three or more votes agreeing,", sum(signals$volume == "CONFIRMED"), "on above-average volume\n")
```

R PRINTED

```
F : 28 signal days, 4 with three or more votes agreeing, 9 on above-average volume
KO : 30 signal days, 6 with three or more votes agreeing, 12 on above-average volume
```

Run it with Code, Source. Run it for the last time. The Console now prints one line for each stock: how many signal days, how many had three or more votes agreeing, and how many were on above-average volume. That is the whole script.

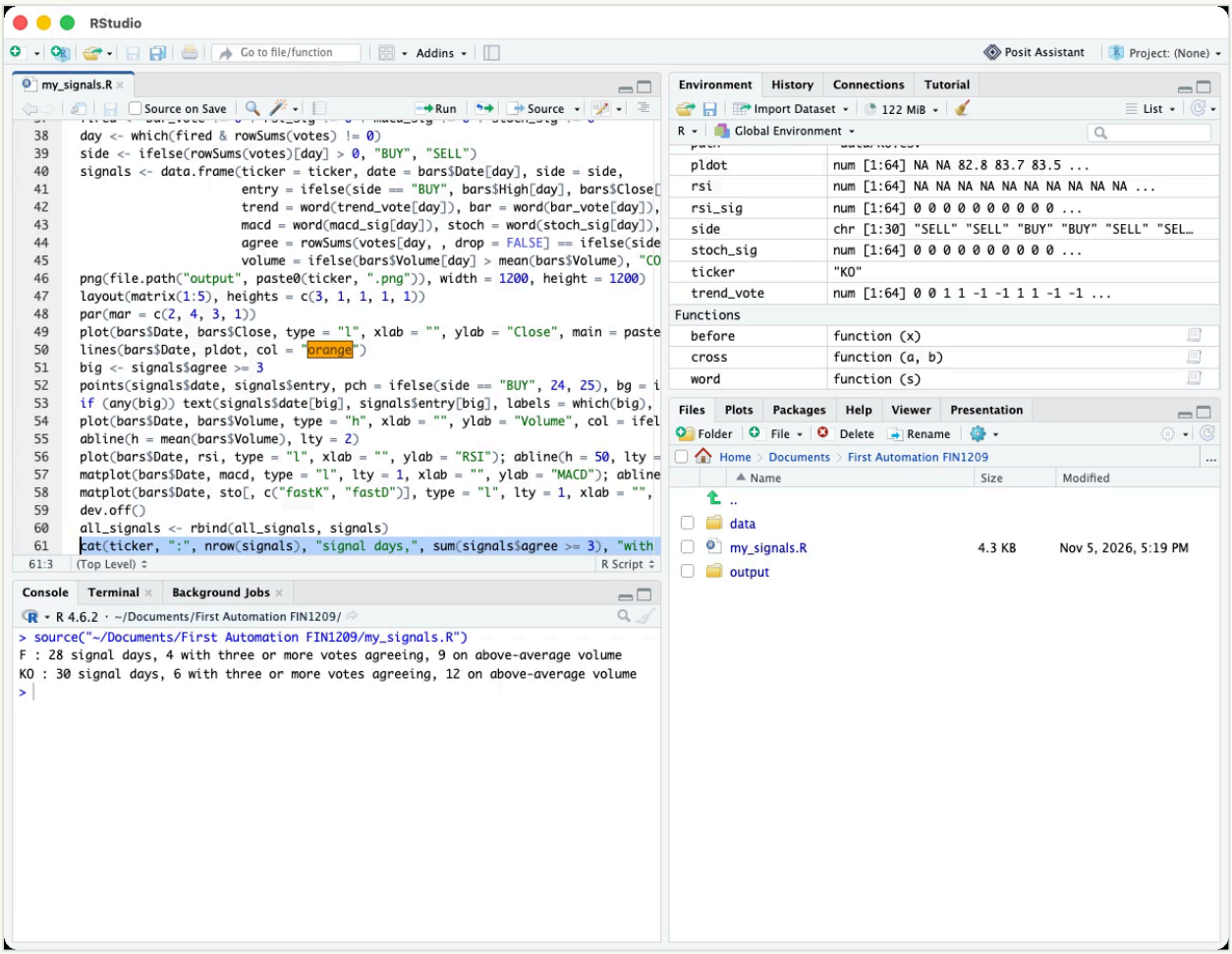
6. Run it and read the output

One answer per stock, a table, a chart. Then new data, and run it again.

1 Run it

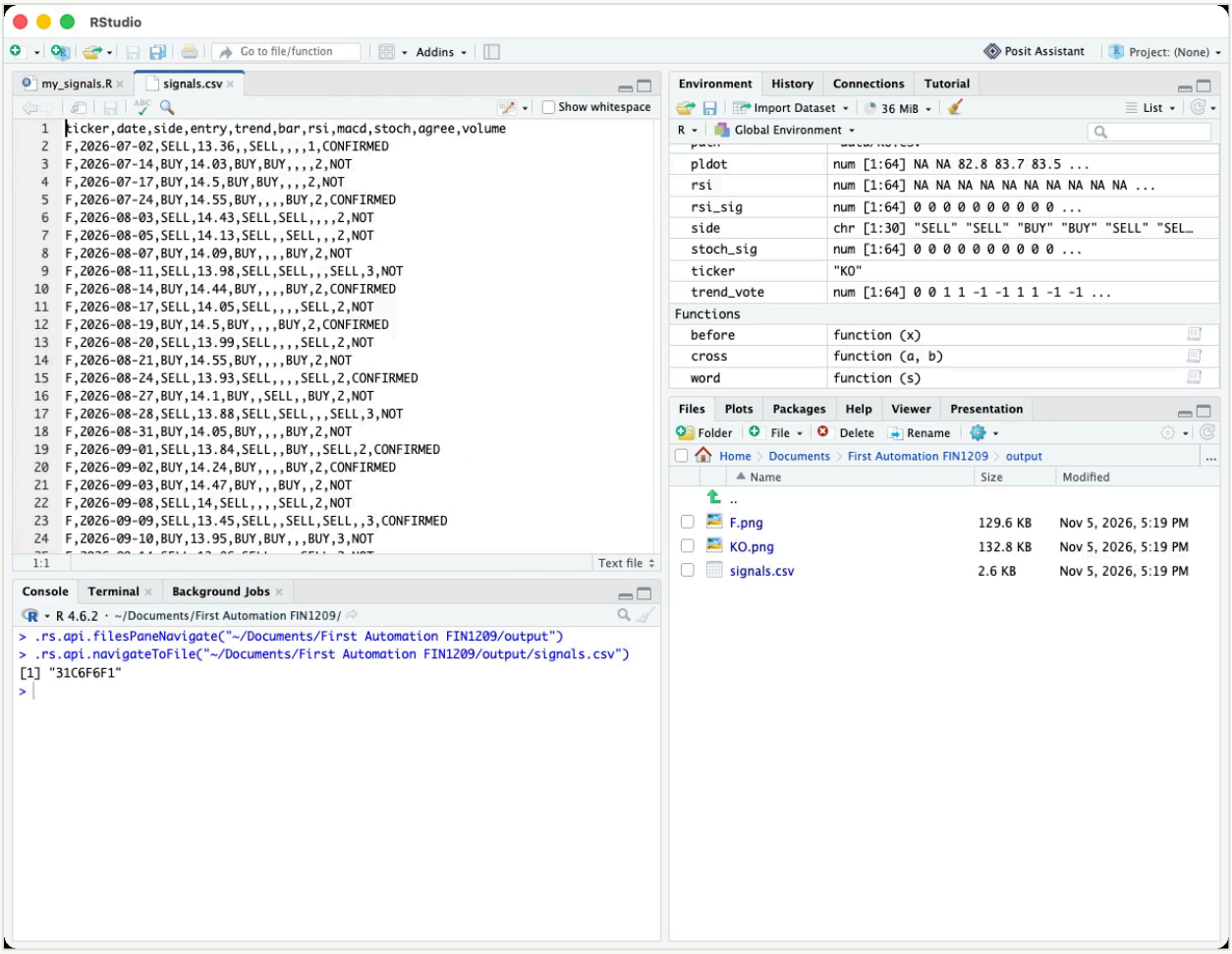
With the finished script open, click **Source**.

The Console prints one line per stock: how many signal days, how many had three or more votes agreeing, and how many were on above-average volume. First-run files: KO 30 signal days, F 28.



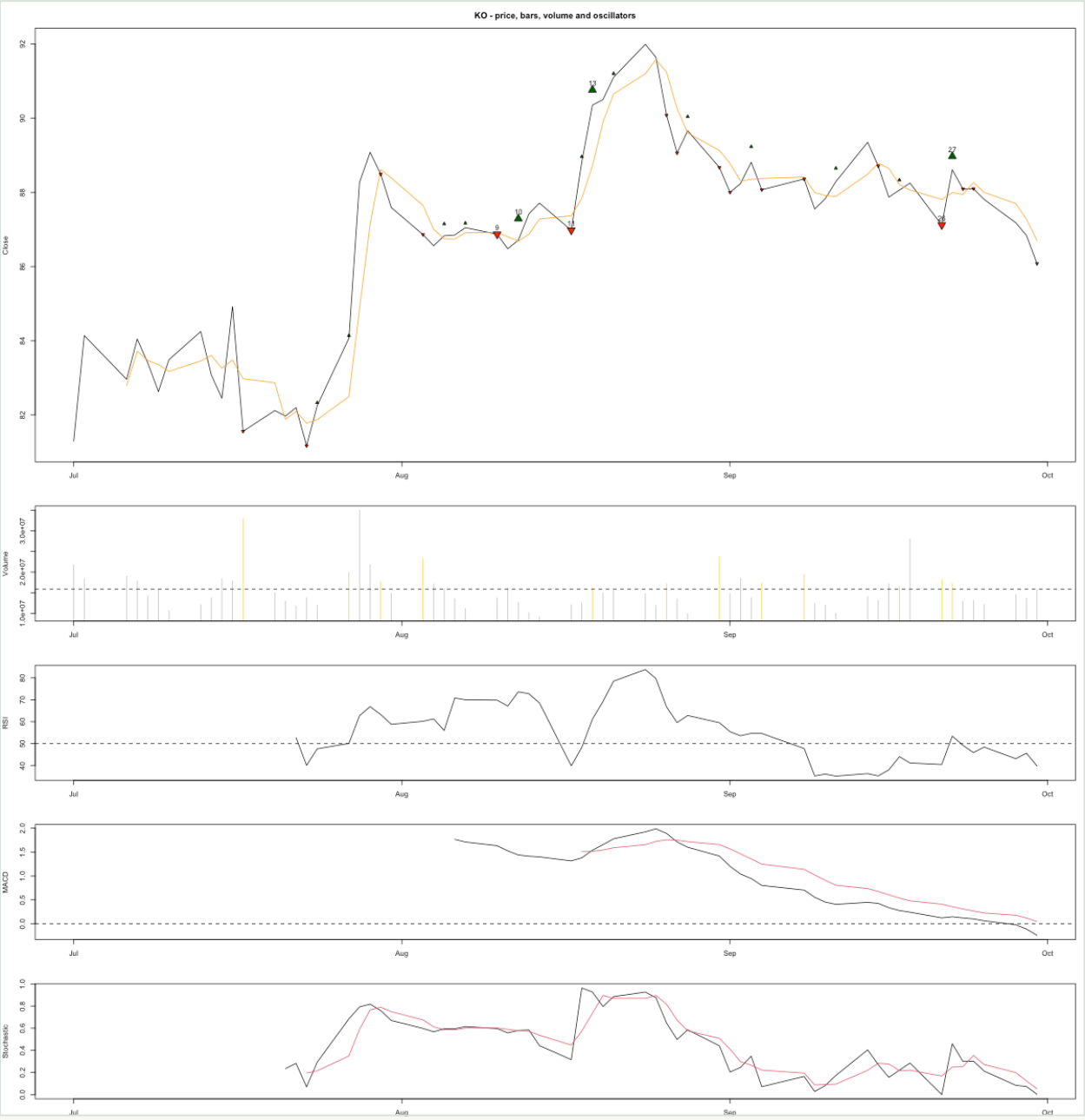
2 Read the table

Open `output/signals.csv`. Each row is a signal day: `side`, the `entry` price, what each vote said (`trend`, `bar`, `rsi`, `macd`, `stoch`), `agree`, and `volume`: CONFIRMED when the day's volume is above the file's average.



3 Read the chart

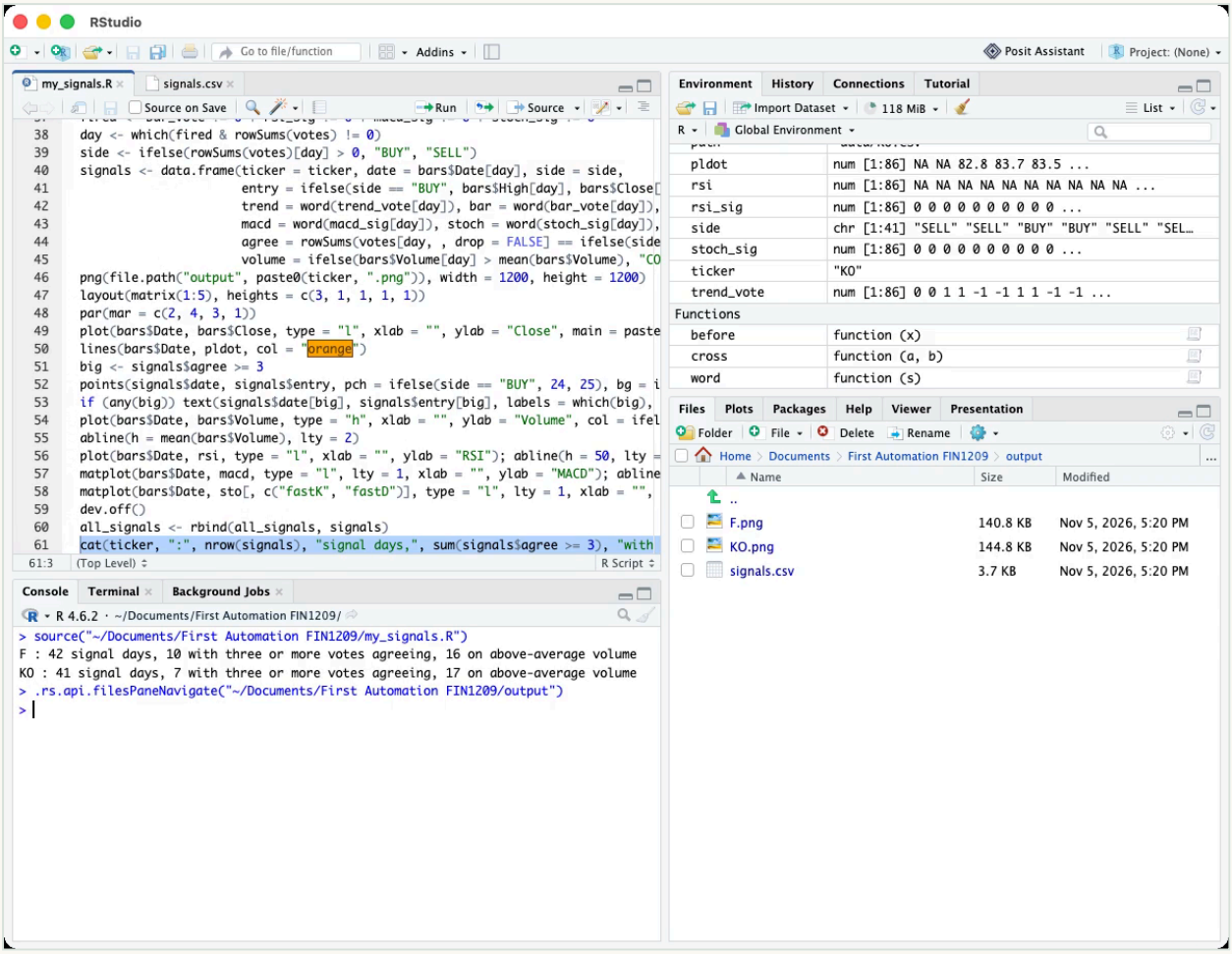
Open `output/KO.png`. Price is on top with its orange three bar average. Every triangle is a signal day, green up for a buy and red down for a sell; the bigger numbered ones are days three or more votes agreed. Below are the volume (gold on CONFIRMED days) and the three oscillators. **The MACD panel is empty at the start:** the MACD needs about 34 days of history before it can give a signal, so the first weeks of this 64 day example show none. For your own stock pull about a year.



4 Add new data and run again

Download both files again with the later end date (Part 2), replace the ones in `data`, and click **Source** again. Nothing else changes.

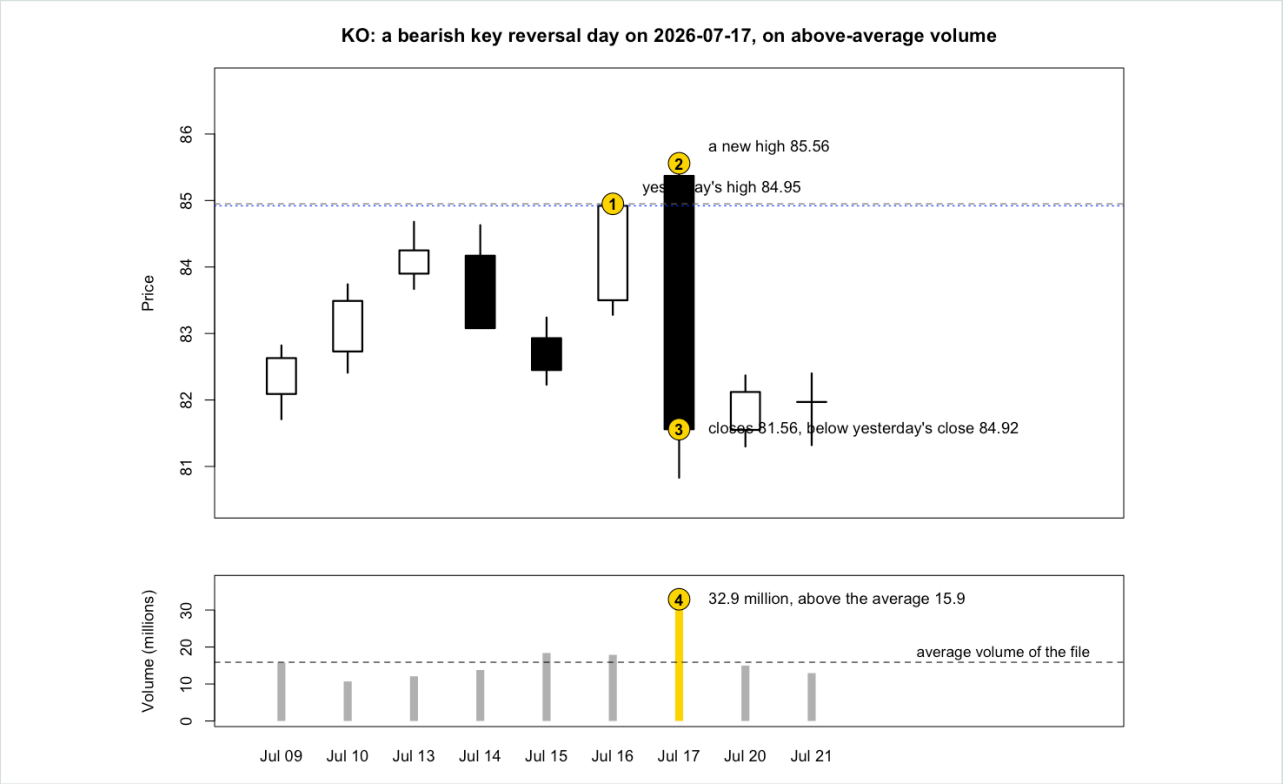
The counts go up: KO from 30 to 41 signal days, F from 28 to 42.



7. Why the signals are where they are

Real signals from the example tables, drawn for the days around them. The script prints numbers only.

A price bar on volume. The bar vote is the key reversal day (slides 123 and 126). In the picture, mark 1 is yesterday's high, mark 2 is today's new high, and mark 3 is the close, below yesterday's close: the buyers pushed price up and could not hold it, so the vote is SELL. Mark 4 is the volume: above the average of the file, so the day is CONFIRMED (slide 128). Volume does not vote. It says how much weight the move carried.

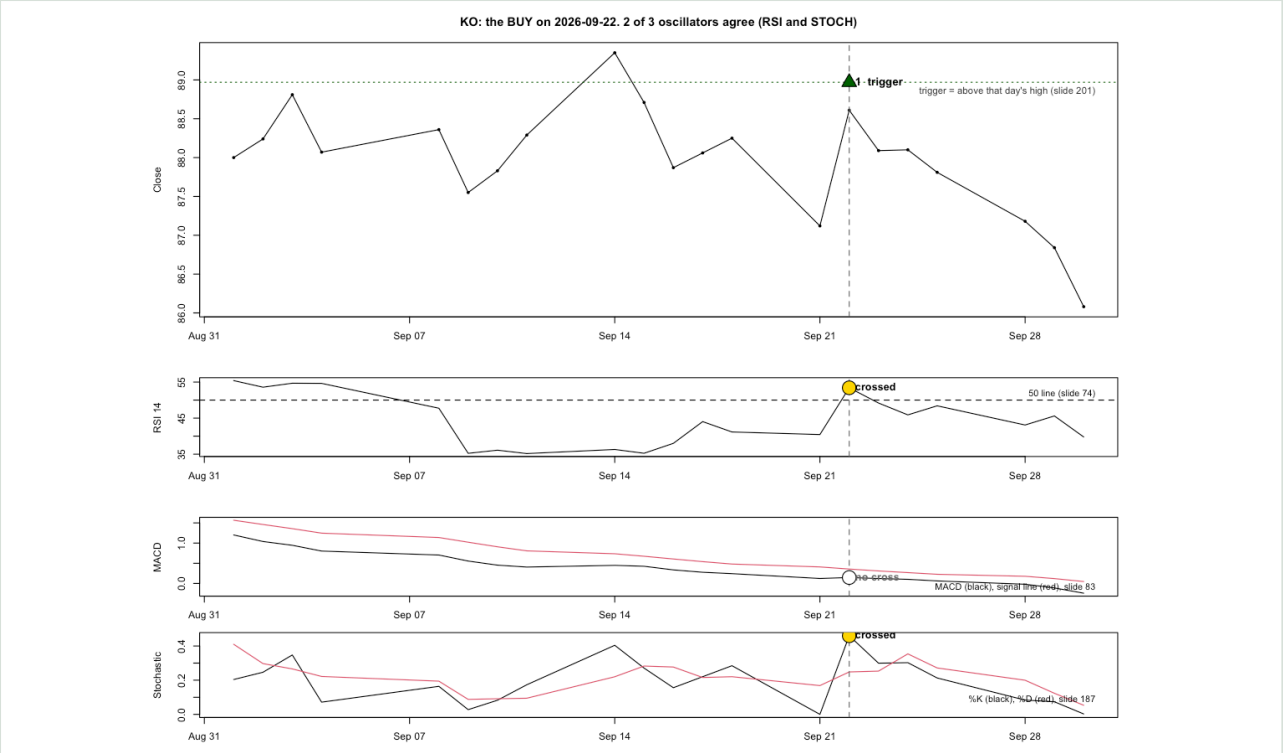


KO, 2026-07-17: a key reversal day, vote SELL, on 32.9 million shares against an average of 15.9.

Drawn by build/r/explain_bar.R from the example price file.

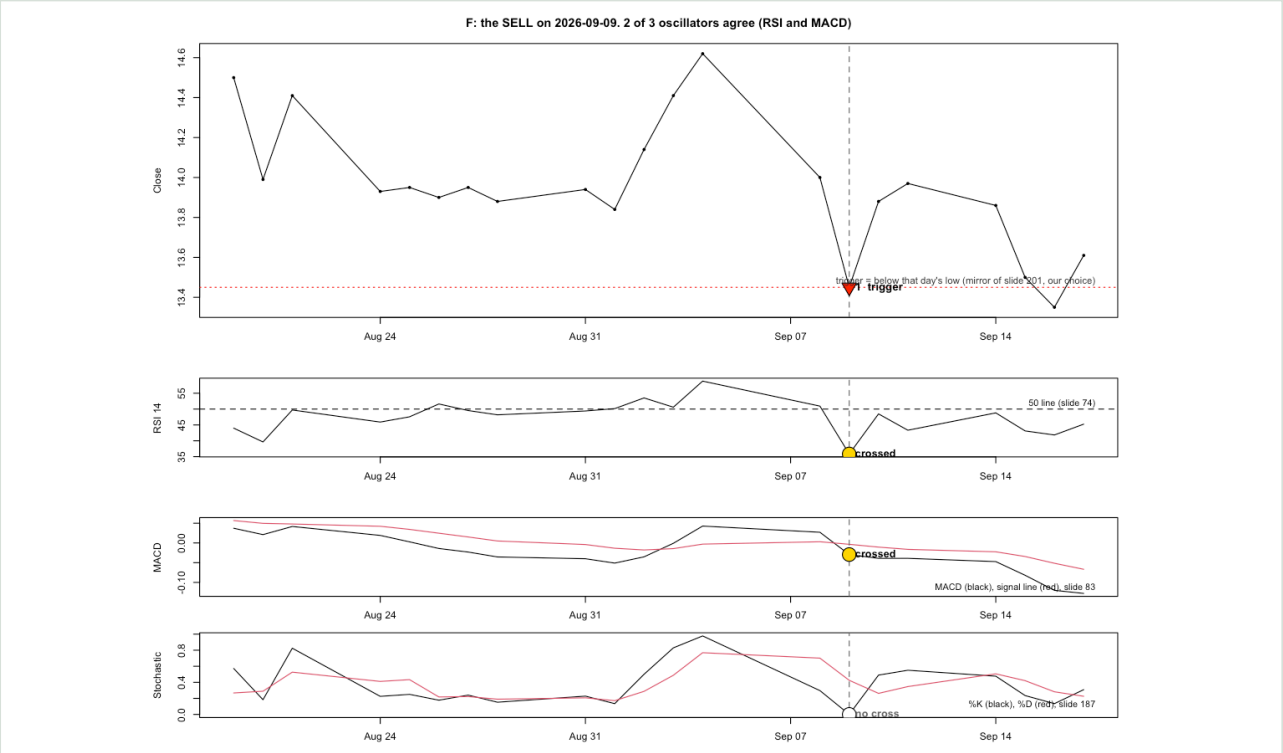
An oscillator buy. On the day the RSI, the MACD or the stochastic crosses upward, that oscillator votes BUY. The trigger is above that day's high (slide 201): the trade happens only if price later goes above it. Gold circles mark the oscillators that crossed and white ones those that did not.

A sell is the mirror image: an oscillator crosses downward. The slides give no sell trigger, so the trigger is below the signal day's low: the mirror of Lim's buy trigger on slide 201, our choice, not Lim's rule. The trade happens only if price later falls below it.



KO, 2026-09-22: a BUY where RSI and STOCH cross. Trigger 88.97.

Drawn by build/r/explain_osc.R from the example price files.



F, 2026-09-09: a SELL where RSI and MACD cross. Trigger 13.45.

Drawn by build/r/explain_osc.R from the example price files.

> AGREEING IS NOT PROOF
>
> Votes agreeing sounds like witnesses agreeing. It is not. The RSI, the MACD and the stochastic are all built from the same closing prices, so they move together: on the example data the RSI and the stochastic have a correlation of 0.73 and the RSI and MACD 0.80. Lim calls this **multicollinearity** and warns that it creates a **false consensus** and a false sense of confidence (slide 78). His cure (slide 81) is to use data other than price, such as volume, which is why the volume check is separate. A signal is only a statement of what a rule says that day, not a forecast.

8. The contract: what every script must do

Your script is marked by running it on files you have never seen, so everybody follows this exactly.

Folders and files

WHAT	RULE
Where it runs	From the folder that holds the script, which holds a folder called `data`.
Input	Every `.csv` in `data` is one stock; the ticker is the file name without `.csv`. At least 60 daily rows each, oldest first. Columns `Date, Open, High, Low, Close, Volume` exactly. `Date` may be `2026-07-02`, `2026-07-02 16:00:00` or `7/2/2026 16:00:00` (Google Sheets).
Output	`output/signals.csv`, every stock together, and one `output/<ticker>.png` per file: price on top, your oscillators below, signal days marked.
Packages	Base R and `TTR`. Nothing else, and no `install.packages` inside the script.
Time	Under two minutes for three files of a year each.

The columns of output/signals.csv, in this order

COLUMN	MEANING	EXAMPLE
`ticker`	The file name without `.csv`	`KO`
`date`	The signal day, `YYYY-MM-DD`	`2026-04-08`
`side`	`BUY` or `SELL`: the side most oscillators gave	`BUY`
`entry`	The trigger level. A buy: the signal day's high. A sell: the signal day's low	`77.37`
`trend, bar`	The two price votes: `BUY`, `SELL` or empty	`BUY`
`rsi, macd, stoch`	One column for each oscillator you use, from `rsi`, `macd`, `stoch`. `BUY`, `SELL` or empty. **Use at least two.** The columns you use tell the marker your pair.	`BUY`
`agree`	How many of your votes gave the `side`	`3`
`volume`	`CONFIRMED` if that day's volume is above the file's average, else `NOT`	`CONFIRMED`

One row per signal day: the bar or an oscillator fires and the votes do not cancel. Rows go ticker by ticker, then by date. The chart number of a day is its row number in the file.

9. Your task

Your own script, on your own stock, and a flowchart of it.

Choose **your own stock** (a US stock, downloaded as in Part 2, about a year of daily rows) and **your own pair** of the three oscillators: RSI and MACD, RSI and stochastic, or MACD and stochastic. Your script keeps the price trend vote, the price bar vote and the volume check, adds your pair, and meets the contract in Part 8. Then draw a **flowchart** of how it works.

> DUE DATE
>
> _____ (to be filled in: after the Saturday 7 November class)

Name your files exactly

FILE	NAME
Your script	`FirstAutomation_Surname_Firstname.R`
Flowchart, PDF (preferred)	`FirstAutomation_Surname_Firstname_Flowchart.pdf`
Or the .bpmn file	`FirstAutomation_Surname_Firstname_Flowchart.bpmn`
Or a Markdown file	`FirstAutomation_Surname_Firstname_Flowchart.md`

Two files, nothing else. No price files or output: they are rebuilt from your script.

Before you submit, check

1. Put your script in a clean folder with a `data` folder of the example files and run it, with no changes.
2. `output/signals.csv` has `ticker, date, side, entry`, your oscillator columns and `agree`, and a `.png` per file is in `output`.
3. Delete `output` and run again. It must come back.
4. Open your flowchart next to your script. Every shape has lines behind it.

> THE MARKER RUNS YOUR SCRIPT ON FILES YOU HAVE NOT SEEN
>
> Real stocks, a different period, in the layout of Part 8. Some files write dates as `2026-07-02`, some as `7/2/2026 16:00:00`. A script that only works on the example files will not pass.

10. Using AI

Allowed, with one condition.

You may use AI to help write your script. The condition is that **you can properly visualise and explain it in your flowchart**. The flowchart is where you show that you understand the script, so:

- The flowchart must match **the script you actually submit**, not an earlier version and not what the AI described.
- Each task must say, in your own words, what that part of the code does, and where the rule has a number, which slide it comes from.
- If you cannot explain a part of the script, do not submit it. Ask for a simpler version you can explain, or go back to the example.

A script that runs perfectly with a flowchart that does not match it loses the flowchart marks. A modest script with a flowchart that matches it and explains every step does well.

11. The marks

100 points. 60 are checked against the record by running your script, and 40 are judged from your flowchart.

Checked against the record

WHAT	MARKS	HOW
The script runs	10	On a folder of files you have never seen, with no changes and no installs, in under 60 seconds.
It writes the files the contract names	10	`output/signals.csv` with the required columns (5), and one `output/<ticker>.png` for every input file (5).
It finds the right signals	20	The ticker, date and side of every signal match the record, with none missing and none extra.
It gets the prices right	10	The trigger level (`entry`) matches the record, to the cent: a buy's is the signal day's high, a sell's is its low.
The chart shows them	10	Each chart is a real chart with every signal marked and numbered.
Subtotal	**60**	

Right or wrong. No rubric is needed because the record decides.

Judged, with the course rubric on the next pages

WHAT	MARKS
The flowchart is correct BPMN	10
The flowchart matches your script	15
The flowchart explains each step	15
Subtotal	**40**

NOT MARKED

How your code looks. Neat or messy, short or long, with comments or without: not marked. Only whether it works and whether the flowchart explains it.

YOUR FLOWCHART IS MARKED ON ITS OWN PAGES, AND THE SIGNALS ARE NOT A FORECAST

The script reports what a rule says at a point in the past. Nothing here is marked on whether a signal would have made money.

How a written answer is marked

For the 40 points that a record cannot check. The same rubric marks every written answer in this course.

> READ THIS FIRST
>
> **We are marking whether your answer is grounded and whether it follows, not how long it is.** A long answer that wanders earns less than two precise sentences.

This rubric is for written answers that have no single correct response. Anything that can be checked against the record, a price, a date, a count, is simply right or wrong, and no rubric is needed for it.

What earns the marks

- **Grounded.** Your answer points at something specific and checkable on your own chart: a price, a date, a shape. Not "the market looks bullish", but "the trough on this date, at this price, is higher than the one before it".
- **Connected.** Your conclusion follows from what you pointed at. It is possible to name three real things and then conclude something that has nothing to do with them.

> HONEST: NAMED, NEVER MARKED
>
> A strong answer also names what argues against it. This is what the best answers do, and it is worth learning to do. **No marks are lost for leaving it out.**

The levels

LEVEL	SHARE OF THE MARKS	WHAT IT LOOKS LIKE
Full	100 percent	Grounded and connected.
Partial	75 percent	One of the two is weak: the evidence is vague, or the conclusion does not quite follow from what was named.
Minimal	50 percent	There is a reason, but it is general rather than taken from your own chart.
None	0 percent	A conclusion with no reasoning, or nothing written.

Percentages rather than marks, so the same rubric marks a question worth four points or forty.

The written answers on this sheet, level by level

ANSWER	OUT OF	FULL, 100%	PARTIAL, 75%	MINIMAL, 50%	NONE, 0%
The flowchart is correct BPMN	10	10	7.5	5	0
The flowchart matches your script	15	15	11.25	7.5	0
The flowchart explains each step	15	15	11.25	7.5	0

Every figure in this table is the level's percentage of what the answer is worth, so it cannot disagree with the levels above.

What the rubric looks for in each of them

ANSWER	GROUNDED HERE MEANS	CONNECTED HERE MEANS
The flowchart is correct BPMN	Points at something checkable in your diagram: a start and an end event, a labelled `yes` and `no` leaving every gateway, an arrow back for every loop.	The shapes are used for what they mean: a diamond is a question, a rectangle is an action, and no path stops without an end event.
The flowchart matches your script	Points at the line of your script each shape stands for, and at every loop and decision in the code having a shape.	Following the arrows reproduces what running the script does, in the same order, with no shape for code that is not there.
The flowchart explains each step	Each task says what its lines do, with the actual column, price or file the code uses, and names the slide where the rule has a price.	The explanation tells the reader why that step is there, not only what it is called.

One question, answered at every level

Here is task 5 of the example flowchart explained five ways, one at each level, and two at Partial because there are two ways to be partial. They are written on the example, not on yours.

LEVEL	THE ANSWER	WHY IT SITS THERE
Full, 100%	Task 5, Add the votes and keep the signal days: it adds the three crossing signals, so a day with one buy and one sell cancels to zero and is skipped. Days with a total above zero are BUYs with entry = High, as slide 201 says, and SELLs with entry = Low, the mirror, and `agree` counts the oscillators that gave that side.	Names what the code adds, a real consequence (cancelling), the exact price rule and the slide.
Partial, 75%	Task 5, Add the votes and keep the signal days: it adds the signals and keeps the days with a signal, using the slide 201 entry price.	Right idea and a slide, but it never says what is added or what `agree` counts, so it cannot be checked.
Partial, 75%	Task 5, Add the votes and keep the signal days: it adds the three signals and keeps the days that are not zero, because three oscillators agreeing means price will rise.	Checkable action, but the reason does not follow: the code does not predict anything.
Minimal, 50%	Task 5, Add the votes and keep the signal days: this is where the program decides which days matter.	A reason, but general: it could describe any task.
None, 0%	Task 5, Add the votes and keep the signal days: votes.	A label with no reasoning.

> WHAT THE BEST ANSWERS ADD
>
> A strong flowchart also says where its script is not Lim's rule: "the slides give no sell trigger, so this script uses the low, the mirror of the buy trigger." It is never marked, and it is the habit that makes a rule you automated one you can trust.